

# Homework #1

CE 6320: Applied Data Structures and Algorithms (Fall 2026)

Department of Electrical and Computer Engineering  
University of Texas at Dallas

**Instructor:** Prof. Mohamed Ibrahim  
**Due Date:** September 13, 2026

**Student Name:** \_\_\_\_\_

**Student ID:** \_\_\_\_\_

## Instructions:

- This homework has **7 parts**.
- You are required to **solve and submit all 7 parts**.
- Each problem will be graded out of its assigned points; your total will be scaled to an overall maximum of **100 points**.
- Show all steps clearly. Partial credit will be awarded.

September 4, 2026

**Problem 1: Circular Linked List — Playlist Rotation**

**[20 points]**

**Problem Description.** A playlist of  $n$  songs labeled  $1..n$  is stored as a *singly circular linked list*. The last node points back to the first node, forming a circle. A pointer `cur` always marks the “current song”. Two basic operations are defined:

- `NEXT(k)`: move `cur` forward by  $k$  steps (wrapping around if needed).
- `DEL()`: delete the node pointed to by `cur`, and set `cur` to the next node.

—

**1. Diagram (5 pts).** Draw a circular linked list for  $n = 5$ . Explain clearly how the nodes are allocated in memory (heap vs stack), how they are connected by next-pointers, and how `cur` points to the “current” node. Illustrate what happens when the node with value 3 is deleted.

—

**2. C++ Implementation (9 pts).** Implement `Node` and fill in the bodies of the following functions using “raw pointers” in C++:

```
// Defines the node of the linked list
struct Node {
    // ...
};

// Builds a circular list of 1..n nodes, returns pointer to head,
// initializes cur appropriately, and sets size = n.
Node* buildCircle(int n, Node*& cur, int& size);

// Advances cur by k % size steps.
void nextK(Node*& cur, int& size, int k);

// Deletes the node at cur, updates links, and sets cur = next node.
void delCurrent(Node*& cur, int& size);

// Prints the list once around starting from cur (or EMPTY if size = 0).
void printOnce(Node* cur);
```

—

**3. Tracking (4 pts).** Start with  $n = 5$  and `cur = 1`. Apply the following sequence of operations:

`NEXT(2), DEL(), NEXT(3), DEL(), NEXT(1).`

After each operation, draw the circular linked list and indicate which node `cur` is pointing to. At the end, show the sequence printed by `printOnce`.

—

**4. Complexity (2 pts).** Compute the time complexity of each function you implemented (`buildCircle`, `nextK`, `delCurrent`, `printOnce`) as a function of  $k$  and  $n$ .

## Problem 2: Unrolled Linked List

[20 points]

**Problem Description.** An *unrolled linked list* is a linked list where each node (called a *block*) stores multiple elements inside a small fixed-size array. This makes the list shorter (fewer pointers) and improves cache performance compared to a regular linked list where each node stores just one element.

Each block can hold up to  $B$  elements. For example, if  $B = 4$  and we want to store the sequence  $[1, 2, 3, 4, 5, 6, 7, 8]$ , the structure looks like this:

$$[1, 2, 3, 4] \rightarrow [5, 6, 7, 8]$$

If we insert one more element (say, 9) into the second block, it overflows. In that case, the block is *split* into two blocks, roughly half-and-half:

$$[1, 2, 3, 4] \rightarrow [5, 6] \rightarrow [7, 8, 9]$$

Similarly, if we delete elements and a block becomes too empty (fewer than half full), it may either *borrow* an element from a neighboring block or *merge* with its neighbor.

We will use 0-based indexing across the entire list: the first element of the first block has index 0, the second has index 1, and so on.

—

**1. Diagram (5 pts).** Let  $B = 4$ . Draw an unrolled linked list that stores the sequence  $[1, 2, 3, 4, 5, 6, 7, 8]$ . Show the blocks, their arrays (with used slots grouped at the front), each block's `cnt` (number of items), and the next-pointers. Explain what lives on the heap (blocks and arrays) versus what lives on the stack (local variables, pointers).

—

**2. C++ Implementation (9 pts).** Using raw pointers and a fixed capacity  $B = 4$ , fill in the function bodies below. Implement insertions with block-split when full, and deletions with borrow/merge when too empty.

```
const int B = 4;

struct Block {
    int a[B];          // items packed in a[0..cnt-1]
    int cnt;           // number of valid items in this block
    Block* next;       // pointer to next block
};

struct UList {
    Block* head;       // head block (nullptr if empty)
    int n;              // total number of items
    UList(): head(nullptr), n(0) {}
};

// Build list from arr[0..n-1], filling blocks left-to-right.
void buildFromArray(const int* arr, int n, UList& L);

// Return the value at global index idx.
```

```
int getAt(const UList& L, int idx);

// Insert x at global index idx, shifting items as needed.
// If a block is full, split it.
void insertAt(UList& L, int idx, int x);

// Erase the element at global index idx, shifting items as needed.
// If a block becomes under half full, borrow or merge with neighbor.
void eraseAt(UList& L, int idx);

// Helper: find block containing index idx, return pointer and local offset.
Block* findBlock(UList& L, int idx, int& off);
```

---

**3. Tracking (4 pts).** Let  $B = 4$ . Start with  $[1, 2, 3, 4, 5, 6, 7, 8]$ . Apply the operations below (indices are global, 0-based):

`insertAt(2, 99), insertAt(6, 77), eraseAt(3), insertAt(8, 55), eraseAt(0).`

After each operation, redraw the blocks showing their contents, counts, and next-pointers. Indicate where a **split**, **borrow**, or **merge** takes place.

---

**4. Complexity (2 pts).** Let the total number of items be  $n$  and block capacity be  $B$ . Give Big- $O$  bounds for:

`buildFromArray, getAt, insertAt, eraseAt.`

Express your answers in terms of  $n$  and  $B$ , and briefly justify each.

### Problem 3: LRU Cache

[20 points]

**Problem Description.** An LRU (Least Recently Used) cache stores up to a fixed number ( $C$ ) of key-value pairs. When the cache is full and a new key must be inserted, the *least recently used* key is evicted. On every successful access (read or write), that key becomes *most recently used*.

In this problem, keys are integers in the range  $[0, K)$ . We implement LRU without hashing by using: (1) a **doubly linked list** that maintains recency order (head = most recent, tail = least recent), and (2) an **array directory** `dir[0..K - 1]` that maps each key to its node (or `nullptr` if absent).

The cache supports:

- `get(key)` → returns the value if present (and moves key to the front), or “not found,”
- `put(key, val)` → inserts/updates; if full and inserting a new key, evict the LRU (tail).

—

**1. Diagram (5 pts).** Draw a memory diagram for a cache with  $K = 10$  and capacity  $C = 3$  after the operations `put(2, 20)`, `put(5, 50)`, `put(7, 70)`. Show:

- the doubly linked list order (head = MRU on the left, tail = LRU on the right),
- which objects live on the heap (nodes, the directory array) vs. on the stack (local variables),
- the directory array `dir` entries that are non-`nullptr` and what they point to.

—

**2. C++ Implementation (9 pts).** Fill in the bodies of the following functions using raw pointers (no hashing). Keys are in  $[0, K)$ .

```
struct Node {
    int key, val;
    Node *prev, *next;
    Node(int k, int v): key(k), val(v), prev(nullptr), next(nullptr) {}
};

struct LRU {
    int cap;           // capacity C
    int size;          // current number of keys in cache
    int K;             // key range upper bound (keys in [0, K))
    Node **dir;        // array directory: dir[key] = Node* or nullptr
    Node *head;        // most recently used (MRU)
    Node *tail;        // least recently used (LRU)
    LRU(): cap(0), size(0), K(0), dir(nullptr), head(nullptr), tail(nullptr) {}
};

// Initialize the cache with capacity Ccap and key range K.
// Allocate dir on the heap and set all entries to nullptr.
void initLRU(LRU &C, int Ccap, int K);

// Return true and set outVal if present (and move node to MRU); else return false.
bool get(LRU &C, int key, int &outVal);

// Insert or update (key, val). If inserting and size==cap, evict LRU.
// Move the (possibly new) key to MRU position.
```

```
void put(LRU &C, int key, int val);

// (Helpers students should implement/and may call from get/put)
// Remove node u from list in O(1).
void detach(LRU &C, Node *u);

// Insert node u at head (MRU) in O(1).
void pushFront(LRU &C, Node *u);

// Evict the LRU (tail) node in O(1); update dir and size; delete the node.
void evictLRU(LRU &C);
```

—

**3. Tracking (4 pts).** Let  $K = 10$  and capacity  $C = 3$ . Starting from an empty cache, apply:

put(2,20), put(5,50), put(7,70), get(2), put(9,90), get(5), put(2,21).

After each operation, show:

- the list from MRU (head) to LRU (tail) as  $[k:v, \dots]$ ,
- which key (if any) was evicted,
- any changes in the directory entries you deem relevant.

—

**4. Complexity (2 pts).** Give tight Big- $O$  time bounds (as functions of  $K$  and  $C$ ) for `initLRU`, `get`, `put`. State the auxiliary space usage (in terms of  $K$  and  $C$ ) and justify briefly.

**Problem 4: Polynomial Representation & Recursive Evaluation**

**[20 points]**

**Problem Description.** We represent a polynomial

$$P(x) = \sum_i c_i x^{e_i}$$

as a *singly linked list* of terms, each term storing a coefficient  $c_i$  and an integer exponent  $e_i$ . Terms are kept in **strictly descending** exponent order (largest exponent first). Example (descending exponents):  $3x^5 + 2x^3 - x + 4$  is stored as the list

$$(3, 5) \rightarrow (2, 3) \rightarrow (-1, 1) \rightarrow (4, 0).$$

When evaluating, note that exponents can be *sparse* (e.g., there is no  $x^2$  term above). A convenient recursive strategy is to “bridge” the exponent gap between consecutive terms using a power of  $x$ . For instance,

$$3x^5 + 2x^3 = (3x^{5-3} + 2) \cdot x^3 = (3x^2 + 2) \cdot x^3.$$

—

**1. Diagram (5 pts).** Using the example  $P(x) = 3x^5 + 2x^3 - x + 4$ , draw the linked list at the level of nodes and pointers. Label each node with (coef,exp). Explain what lives on the heap (nodes) and what lives on the stack (local pointers, temporaries) during typical operations.

—

**2. C++ Implementation (9 pts).** Fill in the bodies of the following functions using raw pointers. Maintain the list in **strictly descending** exponent order with **no duplicate exponents**. When inserting a term whose exponent already exists, **combine** coefficients; if the new coefficient becomes 0, remove that node.

```
struct Term {
    double coef;
    int exp;
    Term* next;
    Term(double c, int e): coef(c), exp(e), next(nullptr) {}
};

// Build a polynomial from arrays coef[i], exp[i], i=0..m-1.
// Insert terms one-by-one so that the final list is strictly descending in exp
// and contains no duplicate exponents (combine coefs; drop zero terms).
Term* buildFromArrays(const double* coef, const int* exp, int m);

// Insert (c,e) into the sorted list (descending exponents).
// Combine coefficients if exp already exists; remove node if coef becomes 0.
void insertTerm(Term*& head, double c, int e);

// Remove the term with exponent e, if present.
void eraseExp(Term*& head, int e);

// Recursively compute x^k for k>=0 using fast exponentiation.
// Must run in O(log k).
double powInt(double x, int k);
```

```
// Recursive evaluation of the polynomial at x.  
// Assume list is strictly descending in exp with no duplicates.  
// Hint: if current node is (c,e) and next is (c2,e2), multiply the running  
// value by x^(e - e2) before adding c2, and recurse.  
double evalRecursive(const Term* head, double x);
```

*Notes.* (i) Do not use `std::vector`, `std::map`, or sorting libraries; work with raw pointers. (ii) You may write small private helpers if needed (e.g., to combine or remove nodes). (iii) Handle corner cases: empty list, single node, insertion at head/tail, deletion at head.

---

**3. Tracking (4 pts).** Start from arrays

`coef = [3, 2, -1, 4],    exp = [5, 3, 1, 0]`

and build  $P(x)$ . Then apply, in order:

`insertTerm(+5, 3),   eraseExp(1),   insertTerm(-3, 5).`

After each operation, redraw the linked list (node contents and arrows), indicating any nodes that were combined or removed. Finally, evaluate the resulting polynomial at  $x = 2$  using your `evalRecursive`. Show the key recursive steps: the exponent gaps you raise  $x$  to, and the partial values as they propagate back up.

---

**4. Complexity (2 pts).** Let  $n$  be the number of *nodes* (nonzero terms) in the list, and let  $\Delta_i$  be the exponent gap between consecutive terms during evaluation. Give tight Big- $O$  bounds for:

`buildFromArrays,   insertTerm,   eraseExp,   evalRecursive.`

Express `evalRecursive` in terms of  $n$  and the cost of powers (i.e.,  $\sum_i \log \Delta_i$  using `powInt`).

### Problem 5: Filesystem Flattening

[20 points]

**Problem Description.** We model a tiny filesystem as a *multilevel linked list*. Each node represents either a directory or a file. Siblings within the same directory are connected by **next**, and a directory's first entry (its sublist of children) is reached via **child**. Your goal is to *flatten* the hierarchy into a single-level list using **next** pointers only, in **preorder** (visit a directory, then its entire subtree, left-to-right), clearing all **child** pointers in the result.

**Example.** Arrows = **next**, downward arrow = **child**:

1 (dir) ↓ (2 (file) → 3 (dir) → 4 (file)), 3 ↓ (5 (file) → 6 (file))

Preorder flattening produces: 1 → 2 → 3 → 5 → 6 → 4, with all **child** pointers set to **nullptr**.

—  
**1. Diagram (5 pts).** Draw the illustrative structure above. Clearly label **next** vs **child** pointers in your diagram. Explain what lives on the heap (nodes) and what lives on the stack (local variables, recursion frames).

—  
**2. C++ Implementation (9 pts).** Fill in the bodies of the following functions using raw pointers. **Requirement:** **flatten must be implemented recursively**, in-place (reuse nodes; do not allocate new nodes just to rearrange). It must produce preorder and set all **child** pointers in reachable nodes to **nullptr** in the result.

```
struct Node {
    int id;           // unique ID (e.g., 1..n)
    bool isDir;       // true = directory, false = file
    Node* next;       // next sibling (or next in flattened list)
    Node* child;      // first child if directory, else nullptr
    Node(int i, bool d): id(i), isDir(d), next(nullptr), child(nullptr) {}
};

// Append 'kid' at the end of directory 'dir' child-list.
void addChild(Node* dir, Node* kid);

// Append 'sib' after node 'u' in the sibling chain (attach at the end of u's next-chain).
void appendSibling(Node* u, Node* sib);

// Recursively flatten the multilevel list rooted at 'head' into a single-level
// preorder list. After flatten, all child pointers in reachable nodes must be nullptr.
// Return the head of the flattened list.
Node* flatten(Node* head);

// Print IDs once along 'next' starting at head (e.g., "10 11 12 ...")
// or "EMPTY" if head==nullptr.
void printFlat(Node* head);
```

—  
**3. Tracking (4 pts).** Build the following multilevel list (IDs shown; arrows = **next**, downward = **child**):

Node 10 (dir)

children: 11 (file)  $\rightarrow$  12 (dir)  $\rightarrow$  13 (file)

12 has children: 14 (dir)  $\rightarrow$  16 (file)

14 has child: 15 (file)

Now call **flatten** on Node 10. After flattening, draw the resulting single-level list (only **next** pointers) and give the exact printout produced by **printFlat**. Then, argue briefly that all **child** pointers reachable from Node 10 are **nullptr** after flattening.

—

**4. Complexity (2 pts).** Let  $n$  be the number of nodes reachable from the head and  $d$  be the maximum directory nesting depth. Give tight Big- $O$  bounds for **addChild**, **appendSibling**, **flatten**, and **printFlat** as functions of  $n$ . State the recursion depth in terms of  $d$ .

**Problem 6: Real-Time Task Scheduling with a Ring Buffer Deque** [20 points]

**Problem Description.** In a small real-time executive, runnable tasks are dispatched in short time slices. To accommodate both urgent and background work, the scheduler uses a *double-ended queue (deque)* so it can insert and remove tasks at either end:

- urgent tasks are enqueued at the *front* (high priority, sooner dispatch),
- background tasks are enqueued at the *back* (lower priority),
- a preempted task may be returned to the *front*,
- the least-important queued task may be dropped from the *back* under memory pressure.

The deque is implemented as a **ring buffer** (circular array) for  $O(1)$  amortized operations and predictable memory use.

**Example.** Let the queue hold `Task{id}` items and start empty with capacity  $C = 5$ . Operations (left = front, right = back):

`ENQ_HIGH(10) ⇒ [10]`, `ENQ_LOW(20) ⇒ [10 | 20]`, `ENQ_HIGH(11) ⇒ [11, 10 | 20]`.

A dispatch step removes from the *front*: `DISPATCH()` yields 11 and the queue becomes `[10 | 20]`. If `PREEMPT(10)` occurs, put 10 back to the *front*: `[10 | 20] → [10, 10 | 20]` (illustrative; in the assignment below, `PREEMPT(id)` enqueues a *given* id at the front).

**1. Diagram (5 pts).** Draw a ring-buffer deque of capacity  $C = 6$  (array indices 0..5) after each operation in the sequence below. Show the positions of **head**, **tail**, and the current **count**. Indicate which indices are occupied and which are free.

`ENQ_LOW(21)`, `ENQ_LOW(22)`, `ENQ_HIGH(11)`, `ENQ_LOW(23)`, `DISPATCH()`, `ENQ_HIGH(12)`, `DROP_LOW()`.

Explain briefly what lives on the heap (the deque array) vs. on the stack (local variables like indices, temporary task objects).

**2. C++ Implementation (9 pts).** Using raw pointers and a single heap-allocated array, fill in the bodies of the following. Your deque must store `Task` objects and support all operations in  $O(1)$  amortized time.

```
struct Task {
    int id;                // unique task identifier
    // you may add small fields if needed (e.g., stamp), but not STL containers
};

struct DequeRB {
    Task* a;               // circular array on the heap
    int cap;               // capacity (fixed at construction)
    int head;              // index of front element (valid iff count > 0)
    int tail;              // index one past the back element (in ring space)
    int count;             // number of elements currently stored

    DequeRB(int C);        // allocate a[C], set fields for empty deque
    ~DequeRB();            // delete[] a

    bool empty() const;
    bool full() const;
```

```
// push to front/back; pre: not full
void pushFront(const Task& t);
void pushBack (const Task& t);

// pop from front/back; pre: not empty
Task popFront();
Task popBack ();

// accessors; pre: not empty
Task& front();
Task& back ();
};

// Simple scheduler facade using the deque.
// ENQ_HIGH -> pushFront, ENQ_LOW -> pushBack,
// DISPATCH -> popFront (returns dispatched Task id)
// PREEMPT(id) -> pushFront(Task{id}),
// DROP_LOW() -> popBack().
struct Scheduler {
    DequeRB q;

    Scheduler(int C);

    void ENQ_HIGH(int id);
    void ENQ_LOW (int id);
    int  DISPATCH();    // returns id of dispatched Task
    void PREEMPT(int id); // place given id back at front
    void DROP_LOW();    // remove one Task from the back
};
```

*Notes.*

- Implement ring arithmetic with modular indices; do not use STL containers.
- When `full()`, you may assume inputs avoid further enqueues unless asked (no need to `resize()`).
- All operations should avoid shifting elements (use index arithmetic to wrap around).

**3. Tracking (4 pts).** Let capacity  $C = 5$ . Starting from an empty scheduler, apply:

`ENQ_LOW(30)`, `ENQ_HIGH(12)`, `ENQ_LOW(31)`, `ENQ_HIGH(13)`, `DISPATCH()`, `PREEMPT(99)`, `ENQ_LOW(32)`, `DROP_LOW`.

After *each* operation:

- draw the ring buffer showing **head**, **tail**, **count**, and occupied indices,
- write the deque order from *front to back* as a list of task IDs,
- for each `DISPATCH()`, report the returned task ID.

**4. Complexity (2 pts).** For your `DequeRB` and `Scheduler`, state tight Big- $O$  bounds (in terms of capacity  $C$  and number of operations  $m$ ) for:

`pushFront`, `pushBack`, `popFront`, `popBack`, `front`, `back`, `ENQ_HIGH`, `ENQ_LOW`, `DISPATCH`, `PREEMPT`, `DROP_LOW`.

Explain briefly and state the auxiliary space used by your implementation.

**Problem 7: Particle Simulation: SoA vs. AoS**

**[20 points]**

**Problem Description.** You are simulating  $N$  point particles in 2D with positions  $(x_i, y_i)$  and velocities  $(v_{x,i}, v_{y,i})$ . At each time step  $\Delta t$ , you advance

$$x_i \leftarrow x_i + v_{x,i} \Delta t, \quad y_i \leftarrow y_i + v_{y,i} \Delta t.$$

Two memory layouts are common:

- **AoS (Array of Structures):** one array of `Particle` where each element stores  $\{x, y, vx, vy\}$  together.
- **SoA (Structure of Arrays):** four parallel arrays `x[]`, `y[]`, `vx[]`, `vy[]` storing each field separately.

**Example.** Let  $N = 3$ . AoS stores:

$$P[0] = \{x_0, y_0, vx_0, vy_0\}, \quad P[1] = \{x_1, y_1, vx_1, vy_1\}, \quad P[2] = \{x_2, y_2, vx_2, vy_2\}.$$

SoA stores:

$$\mathbf{x} = [x_0, x_1, x_2], \quad \mathbf{y} = [y_0, y_1, y_2], \quad \mathbf{vx} = [vx_0, vx_1, vx_2], \quad \mathbf{vy} = [vy_0, vy_1, vy_2].$$

A single update step adds  $\Delta t \cdot vx[i]$  to `x[i]` and  $\Delta t \cdot vy[i]$  to `y[i]` for each  $i$ . This assignment compares the two layouts in code, tracing, and complexity.

—

**1. Diagram (5 pts).** Draw both layouts for  $N = 4$ :

- (a) **AoS:** show the contiguous array of `Particle` elements and the field order inside each element.
- (b) **SoA:** show the four separate arrays and how index  $i$  refers to one particle across arrays.

For each layout, explain what is allocated on the **heap** (arrays and/or array of structs) and what lives on the **stack** (local pointers, loop indices, temporaries).

—

**2. C++ Implementation (9 pts).** Using raw pointers (no STL containers), fill in the bodies of the following functions for *both* AoS and SoA. Assume doubles for all components, and that memory is heap-allocated in the `init/build` functions.

```
// ----- AoS -----
struct Particle {
    double x, y, vx, vy;
};

struct AoS {
    Particle* p;    // array length N on the heap
    int N;
};

// Allocate AoS of size N and initialize from arrays (x[], y[], vx[], vy[]).
void aosInit(AoS& A, int N, const double* x, const double* y,
             const double* vx, const double* vy);

// Advance positions by dt: x += vx*dt, y += vy*dt for all particles.
```

```
void aosStep(AoS& A, double dt);

// Compute center of mass: returns (cx, cy) via refs.
void aosCenterOfMass(const AoS& A, double& cx, double& cy);

// Free AoS heap memory.
void aosFree(AoS& A);

// ----- SoA -----
struct SoA {
    double *x, *y, *vx, *vy; // four arrays length N on the heap
    int N;
};

// Allocate SoA of size N and initialize from arrays (x[], y[], vx[], vy[]).
void soaInit(SoA& S, int N, const double* x, const double* y,
             const double* vx, const double* vy);

// Advance positions by dt: x[i] += vx[i]*dt, y[i] += vy[i]*dt.
void soaStep(SoA& S, double dt);

// Compute center of mass: returns (cx, cy) via refs.
void soaCenterOfMass(const SoA& S, double& cx, double& cy);

// Free SoA heap memory.
void soaFree(SoA& S);

// ----- Layout transform (optional helper pair) -----
// Copy A -> S (same N); Copy S -> A (same N). Assume targets allocated.
void aosToSoa(const AoS& A, SoA& S);
void soaToAos(const SoA& S, AoS& A);
```

*Notes.* (i) Assume inputs are valid and memory allocation succeeds. (ii) Use simple for-loops; do not use SIMD or libraries. (iii) Avoid temporary allocations inside tight loops.

### 3. Tracking (4 pts).

Let  $N = 3$  with initial arrays

$$\mathbf{x} = [1, -2, 0], \quad \mathbf{y} = [0, 3, -1], \quad \mathbf{vx} = [2, -1, 4], \quad \mathbf{vy} = [-3, 0, 1], \quad \Delta t = 0.5.$$

- (a) Show the AoS memory picture *before* the step and *after* one call to `aosStep(A, dt)` (values for all fields).
- (b) Show the SoA memory picture *before* the step and *after* one call to `soaStep(S, dt)` (values in each array).
- (c) Using your functions, compute and report the center of mass ( $c_x, c_y$ ) after the step for each layout.

**4. Complexity (2 pts).** Let  $N$  be the number of particles. State the time complexity and auxiliary space of each function (`aosInit`, `aosStep`, `aosCenterOfMass`, `aosFree`, `soaInit`,

`soaStep`, `soaCenterOfMass`, `soaFree`, and optionally `aosToSoa/soaToAos`) as functions of  $N$ . Briefly explain any differences in memory access behavior between AoS and SoA for the `Step` and `CenterOfMass` operations.

- - - - - END - - - - -